

Asignaturas: Análisis y Diseño de Software

Sistema de Gestión Vehicular “AutoDrive Motors”

Enunciado de la Problemática encontrada en la empresa:

La empresa de venta de vehículos “AutoDrive Motors” actualmente administra su información de manera manual mediante hojas de cálculo y documentos físicos.

Debido al crecimiento de la empresa, se han presentado múltiples problemas en el manejo de la información, entre ellos:

- pérdida de datos de clientes
- errores en los precios de los vehículos
- ventas duplicadas
- dificultad para conocer qué vehículos están disponibles
- problemas para consultar historiales de mantenimiento
- retrasos en la atención al cliente
- falta de control sobre los vehículos vendidos
- dificultad para generar reportes rápidos

Además, la gerencia desea que el sistema pueda consultar información externa relacionada con tasas de cambio para convertir automáticamente el valor de los vehículos de pesos colombianos a dólares.

La empresa requiere el desarrollo de una solución tecnológica basada en API REST utilizando Spring Boot, una base de datos relacional y pruebas mediante Postman.

Los estudiantes deberán analizar la problemática, diseñar la solución e implementar el backend del sistema aplicando buenas prácticas de desarrollo.

Objetivo General

Desarrollar una API RESTful para la gestión de vehículos, clientes, ventas y mantenimientos de la empresa AutoDrive Motors, implementando conexión a base de datos y consumo de API externa.

Requerimientos Funcionales

El sistema debe permitir:

- Gestión de Clientes
- Registrar clientes
- Consultar clientes
- Actualizar información
- Eliminar clientes

Gestión de Vehículos

- Registrar vehículos
- Consultar vehículos
- Actualizar vehículos
- Eliminar vehículos
- Consultar vehículos por marca
- Consultar vehículos disponibles

Gestión de Ventas

- Registrar ventas
- Asociar cliente y vehículo
- Calcular total de venta
- Cambiar automáticamente el estado del vehículo a “Vendido”

Gestión de Mantenimientos

- Registrar mantenimientos
- Consultar historial de mantenimientos
- Cambiar estado del vehículo a “En mantenimiento”

Consumo de API Externa

El sistema deberá consumir una API pública para:

- consultar la tasa de cambio actual
- convertir automáticamente el valor del vehículo de COP a USD

Requerimientos Técnicos

Los estudiantes deben utilizar:

- Java
- Spring Boot
- JPA / Hibernate
- MySQL o PostgreSQL
- Arquitectura por capas
- DTOs
- Validaciones
- Manejo de excepciones
- Postman

Reglas de Negocio

El sistema debe cumplir las siguientes reglas:

1. No se puede vender un vehículo que esté:

- vendido
- en mantenimiento

2. No se permiten precios negativos.

3. La placa del vehículo debe ser única.

4. Los clientes no pueden registrarse con correos repetidos.

5. Toda venta debe generar automáticamente:

- fecha de venta
- total de venta

6. Si el valor del vehículo supera los \$100.000.000 COP:

- aplicar descuento del 5%

Requerimientos de Diseño y Análisis

Los estudiantes deberán entregar:

- Diagramas UML
- Casos de uso
- Diagrama de clases
- Diagrama de secuencia

Base de Datos

- Modelo relacional
- DER (Diagrama Entidad Relación)

Documentación

- Requerimientos funcionales
- Requerimientos no funcionales
- Historias de usuario

Endpoints Mínimos Esperados

- Clientes
- GET /clientes
- POST /clientes
- PUT /clientes/{id}
- DELETE /clientes/{id}

Vehículos

- GET /vehiculos
- POST /vehiculos
- GET /vehiculos/disponibles
- GET /vehiculos/marca/{marca}

Ventas

- POST /ventas
- GET /ventas

Mantenimientos

- POST /mantenimientos
- GET /mantenimientos

Se debe entregar la solución funcional que incluya:

- 1) API REST funcionando
- 2) Base de datos relacional conectada
- 3) Relaciones entre entidades
- 4) CRUD completos
- 5) Validaciones implementadas
- 6) Consumo de API externa
- 7) Pruebas en Postman
- 8) Arquitectura organizada por capas
- 9) Manejo de excepciones
- 10) DTOs
- 11) Documentación UML y DER

Nivel Intermedio Esperado

- La solución debe evidenciar:
- uso correcto de relaciones JPA
- lógica de negocio
- consumo de APIs externas
- separación de responsabilidades
- buenas prácticas de backend
- manejo de JSON
- validaciones robustas